Язык программирования Python

Наследование

Доцент кафедры ИВТ, к.т.н.
Проскурин Александр Викторович

Наследование

Наследование – это отношение, связывающее классы, один из которых является **базовым** (**родительским**), а другой создается на его основе (**производный**, **наследник**).

Наследование заключается в том, что класс-наследник приобретает поля и методы родительского класса и добавляет к ним собственные. Он так же может переопределять (т.е. реализовывать иначе) методы родительского класса.

Таким образом, возможно создание **иерархии классов**, в которой базовые классы обладают более общими элементами, а производные классы их конкретизируют (расширяют).

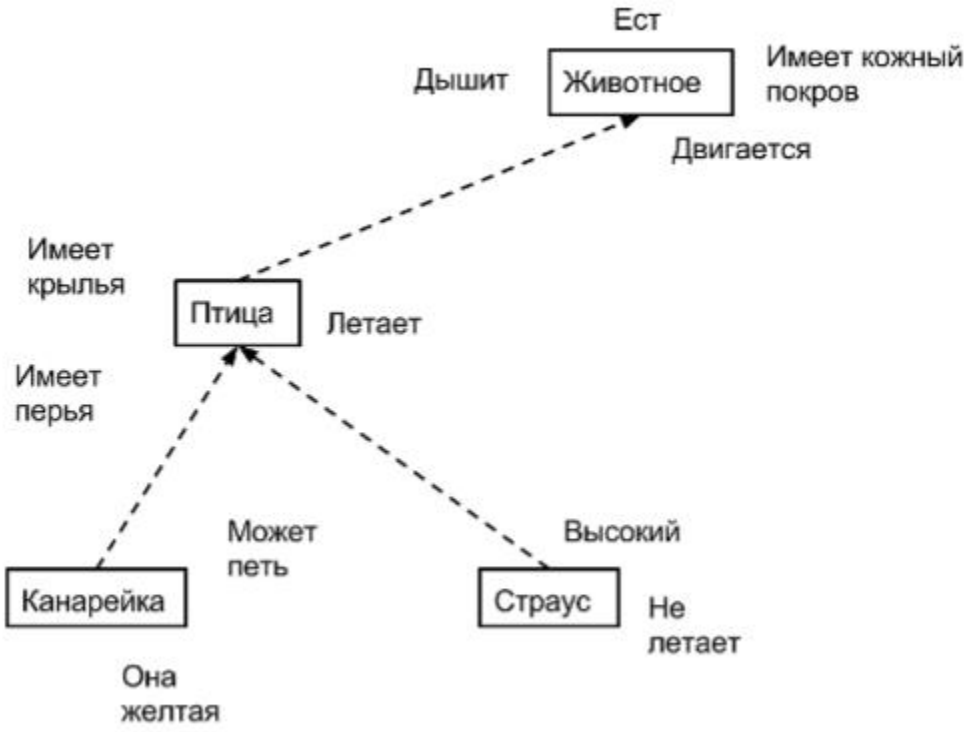
В свою очередь производный класс может служить базовым для другого класса.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Преимущества наследования

Принципы наследования неявно применяются при запоминании информации об окружающем мире.

Так, вместо запоминания всех свойств каждой отдельной птицы, мы запоминаем только её отличительные черты (для канарейки это умение петь и жёлтый цвет), перенося общие свойства (такие как умение летать и наличие крыльев) на более абстрактное понятие «птица».



Эксперименты показали, что ответ на вопрос «Может ли канарейка петь?» требует меньше времени, чем на вопрос «Может ли канарейка дышать?»

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Объявление производного класса

Основная форма синтаксиса объявления производного класса следующая:

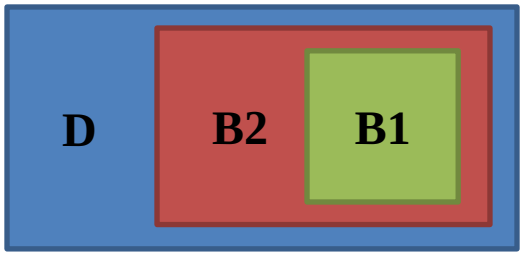
```
class Имя_производного_класса (Имя_базового_класса):
    """ Описание класса """
    def __init__(self, параметр_базового1, параметр1):
        # Вызов конструктора базового класса.
        # Необходим для создания и присваивания
        # значений полям базового класса. Параметров
        # может быть несколько или ни одного.
        super().__init__(параметр_базового1)
        # Добавление нового поля в производный класс
        # (новых полей может быть несколько или ни
        # одного).
        self.поле1 = параметр1

    др_методы_производного_класса
```

```
class A:
    #...
class X(A):
    #...
# A – базовый класс
# X – производный класс
```

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Вызов конструкторов (1)

В конструкторе производного класса нужно явным образом вызывать конструктор базового класса. В противном случае поля базового класса не будут созданы.




```
class B1:
    def __init__(self):
        print('Раб. конструктора B1')

class B2(B1):
    def __init__(self):
        super().__init__()
        print('Раб. конструктора B2')

class D(B2):
    def __init__(self):
        super().__init__()
        print('Раб. конструктора D')

if __name__ == '__main__':
    d = D()
```

Вызов конструктора базового класса

Вызов конструктора базового класса

Результаты работы программы:

```
Раб. конструктора B1
Раб. конструктора B2
Раб. конструктора D
```

Выполнять код только если он был запущен, а не импортирован

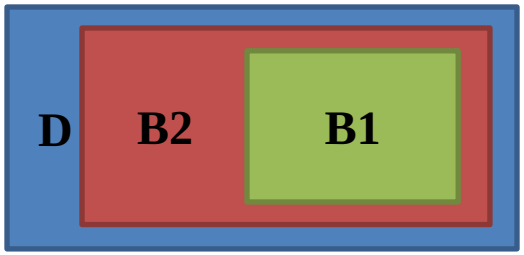
© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Вызов конструкторов (2)

Если подкласс не определяет свой конструктор, то он унаследует конструктор родительского класса (т.е. автоматически будет вызываться конструктор базового класса).

```
class B1:
    def __init__(self):
        print('Раб. конструктора B1\n')

class B2(B1):
    def __init__(self):
        super().__init__()
        print('Раб. конструктора B2\n')

class D(B2):
    pass

if __name__ == '__main__':
    d = D()
```

pass означает, что реализация отсутствует. Написано для упрощения примера.



Результаты работы программы:

```
Раб. конструктора B1
Раб. конструктора B2
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Доступ к атрибутам класса

В Python действует соглашение о разделении доступа к атрибутам класса на закрытые, защищенные и открытые в зависимости от того, где предполагается использовать тот или иной атрибут.

Спецификатор	Значение. Особенность написания
private (закрытый / собственный)	Атрибуты могут быть использованы только методами класса. Перед именем атрибута ставится два символа нижнего подчёркивания (__).
protected (защищенный)	Предполагает, что атрибуты могут использоваться методами самого класса, а также наследниками данного класса. Перед именем атрибута ставится один символ нижнего подчёркивания (_).
public (открытый / общедоступный)	Атрибуты доступны всем функциям (коду), в том числе и не методам класса. Перед названием атрибута ничего не ставится.

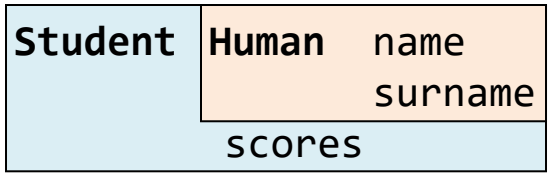
© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример наследования (1)

Предположим, нужно хранить информацию о преподавателях и студентах.

Мы можем создать два независимых друг от друга класса, но это будет неэффективно, т.к. придётся дублировать общие поля и методы. Вместо этого мы выделим их в общий класс и унаследуем от него два других.

```
class Human: # Базовый класс, моделирует абстрактного человека
    def __init__(self, name, surname):
        self.__name = name          # У него есть имя
        self.__surname = surname    # и фамилия
    # Метод возвращает полное имя
    def get_full_name(self):
        return self.__name + ' ' + self.__surname

class Student(Human): # Производный класс, моделирует студента
    def __init__(self, name, surname, scores):
        super().__init__(name, surname) # Помимо имени и фамилии
        self.__scores = scores          # у него есть оценки
    # Вычислим среднее значение оценок
    def get_avg_score(self):
        sum_scores = sum(self.__scores.values())
        return sum_scores / len(self.__scores)
```

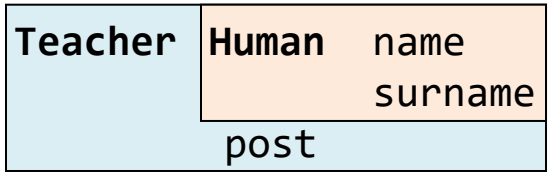


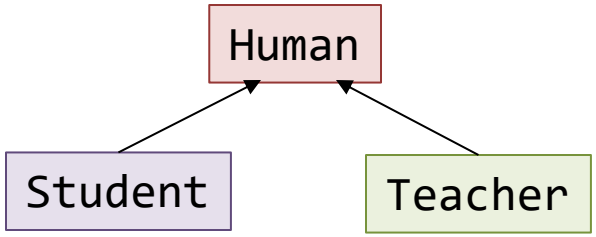
Предполагается словарь

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример наследования (2)

```
class Teacher(Human): #Производный класс, моделирует преподавателя
    def __init__(self, name, surname, post):
        super().__init__(name, surname) # Помимо имени и фам.
        self.__post = post              # у него есть должность
    # Возвращаем название должности
    def get_post(self):
        return self.__post
```






```
if __name__ == '__main__':
    ch = Human('Николай', 'Иванов')
    print('Человек', ch.get_full_name())

    st = Student('Фёдор', 'Петров', {'ООП':5, 'БД':4})
    print('Студент', st.get_full_name(), # Метод базового и
          'имеет среднюю оценку', st.get_avg_score()) # производ.

    tc = Teacher('Иван', 'Николаев', 'Доцент')
    print('Преподаватель', tc.get_full_name(), # Метод базового и
          'имеет должность', tc.get_post()) # производного классов
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Доступ к наследуемым атрибутам

Попробуем добавить в класс Student метод info() с вызовом функции print:

```
def info(self):
    # нет доступа к закрытым полям базового класса
    print('Студент', self.__surname, self.__name) # Ошибка!

    # мы можем вызывать открытые методы базового класса
    print('Студент', self.get_full_name()) # ок

    # а также любые методы своего класса
    print('Средняя оценка: ', self.get_avg_score())
```

Важно! Закрытый элемент класса доступен **только** для других атрибутов **этого класса**.

Для прямого обращения к **атрибутам базового** класса необходимо написать их имя с одним символом нижнего подчёркивания (**доступ protected)** или использовать **общедоступные методы** базового класса.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример прямого доступа к наследуемым атрибутам

```
class Human: # Базовый класс
    def __init__(self, name, surname):
        # Обратите внимание на один символ нижнего
        # подчёркивания. Теперь мы можем обращаться
        # к полям базового класса напрямую
        self._name = name
        self._surname = surname

class Student(Human): # Производный класс
    def __init__(self, name, surname, scores):
        super().__init__(name, surname) # Здесь ничего
        self.__scores = scores          # не меняем

    def info(self):
        print('Студент', self._surname, self._name) # ок
        print('Имеет следующие оценки:')
        for subject, score in self.__scores.items():
            print(f'{subject:5} {score}')

if __name__ == '__main__':
    st = Student('Фёдор', 'Петров', {'ООП':5, 'БД':4})
    st.info()
```

```
Студент Петров Фёдор
Имеет следующие оценки:
ООП   5
БД    4
```

Обращение к полям базового класса

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перегрузка методов при наследовании

Одним из преимуществ объектно-ориентированного программирования является возможность для методов базового класса задать в производных от него классах отличающуюся реализацию (**полиморфизм**).

В этом случае имя метода будет совпадать в базовом и производных классах, но будут отличаться выполняемые ими действия.

Если внутри метода какого-то производного класса необходимо вызвать перегруженный метод базового класса, то используется следующий синтаксис:

```
super().имя_метода(параметры)
```

super – это ключевое слово, которое используется для обращения к родительскому классу. Если заданное имя метода не будет найдено непосредственно в базовом классе, то поиск продолжится в базовом классе базового класса («дедушке») и т.д.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример вызова перегруженного метода базового класса

```
class Human: # Базовый класс
    def __init__(self, name, surname):
        self._name = name
        self._surname = surname

    def info(self, title = 'Человек'):
        print(title, self._name, self._surname)

class Student(Human): # Производный класс
    def __init__(self, name, surname, scores):
        super().__init__(name, surname)
        self.__scores = scores

    def info(self, title = 'Студент'):
        super().info(title)
        print('Имеет следующие оценки:')
        for subject, score in self.__scores.items():
            print(f'{subject:5} {score}')

if __name__ == '__main__':
    st = Student('Фёдор', 'Петров', {'ООП':5, 'БД':4})
    st.info()
```

```
Студент Петров Фёдор
Имеет следующие оценки:
ООП   5
БД    4
```

Вызов метода info() базового класса Human

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Абстрактные классы (1)

Абстрактные классы предназначены **для представления общих понятий**, которые предполагается конкретизировать в производных классах.

При создании иерархии классов часто бывает логично выделить их общие свойства в один родительский класс. При этом может оказаться, что создавать экземпляры такого класса не имеет смысла, потому что никакие реальные объекты им не соответствуют. Такие классы называют абстрактными.

Абстрактные классы служат только в качестве базовых классов. **Создать экземпляр абстрактного класса нельзя.** Как правило, в абстрактных классах создается набор методов, которые каждый из производных классов будет реализовывать по-своему.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Абстрактные классы (2)

Абстрактный класс – это класс с одним или большим количеством абстрактных методов (объявленные, но не реализованные). Помимо абстрактных методов могут быть и обычные.

Для его создания необходимо импортировать модуль abc (Abstract Base Class). Синтаксис использования:

```
from abc import ABC, abstractmethod
class Имя(ABC): # Наследуем абстрактный класс от класса ABC
    # Конструктор объявляется как обычно (если нужны поля)
    def __init__(self, параметр1...):
        # Инициализация полей

    # Объявляем абстрактные методы
    @abstractmethod # Сообщаем, что он именно абстрактный
    def имя_метода(self):
        pass # Ничего не делаем (нет реализации)

    другие_методы_класса # Могут быть обычными
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример абстрактного класса (1)

```
from abc import ABC, abstractmethod
import math

class Shape(ABC): # Абстрактный класс геометрической фигуры
    # абстрактный метод, который будет необходимо
    # переопределять для каждого производного класса
    @abstractmethod
    def area(self): # Для вычисления площади фигуры
        pass # нет реализации

    def my_name(self): # Обычный метод
        return self.__class__.__name__ # Вернёт имя класса

# Производный класс Круг с реализацией абстрактного метода
class Circle(Shape):
    def __init__(self, radius):
        self.__radius = radius # радиус

    def area(self):
        return math.pi * self.__radius**2
```

Так можно узнать имя класса

my_name реализовывать не нужно, т.к. базовая реализация достаточна

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример абстрактного класса (2)

```
class Rectangle(Shape): # Производный класс Прямоугольник
    def __init__(self, height, width):
        self.__height = height # высота
        self.__width = width   # ширина
    def area(self):
        return self.__height * self.__width

if __name__ == '__main__': # Основная программа
    p = [] # Будем заполнять список из кругов и прямоугольн.
    for _ in range(5):  # Всего 5 штук с клавиатуры
        t = int(input(f'Введите номер типа (0 - Круг, '
                      f'1 - Прямоугольник): '))
        if t == 0: # Если выбрали Круг
            r = float(input('Введите радиус: '))
            p.append(Circle(r))
        elif t == 1: # Если выбрали Прямоугольник
            h = float(input('Введите высоту: '))
            w = float(input('Введите ширину: '))
            p.append(Rectangle(h, w))
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример абстрактного класса (3)

```
# Продолжение основной программы
# Вычисляем суммарную площадь всех экземпляров
ttl_area = 0
for i in p: # В списке круги и прямоугольники вперемешку
    ttl_area += i.area() # но у всех есть метод area()
print('Площадь всех фигур = ', ttl_area)

# находим среднюю площадь объектов класса Circle
crc_avg = 0
num_crc = 0
for i in p: # Круги выбираем по имени класса
    if i.my_name() == 'Circle':
        crc_avg += i.area()
        num_crc += 1
print('Средняя площадь кругов = ', crc_avg / num_crc)

print('Создать экземпляр абстрактного класса нельзя!')
s = Shape()
```

TypeError: Can't instantiate abstract class Shape with abstract methods area

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Множественное наследование

Множественное наследование – производный класс наследуется от двух и более базовых классов. В случае вызова конструкторов и других переопределённых методов нужно явно указать имя базового класса.

```
class Имя_производного_класса (Имя_баз_класса_1,
                               Имя_баз_класса_2,
                               ...):
    """ Описание класса """
    def __init__(self, параметр_баз1,
                 параметр_баз2, параметр1):
        # Вызывать конструкторы базовых классов нужно
        # с помощью имён этих классов.
        Имя_баз_класса_1.__init__(self, параметр_баз1)
        Имя_баз_класса_2.__init__(self, параметр_баз2)
        # Добавление нового поля в производный класс
        self.поле1 = параметр1

    др_методы_производного_класса
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример множественного наследования

Есть два класса Human (человек) и Employee (работник). Необходимо создать класс Teacher (учитель), который является наследником обоих классов.

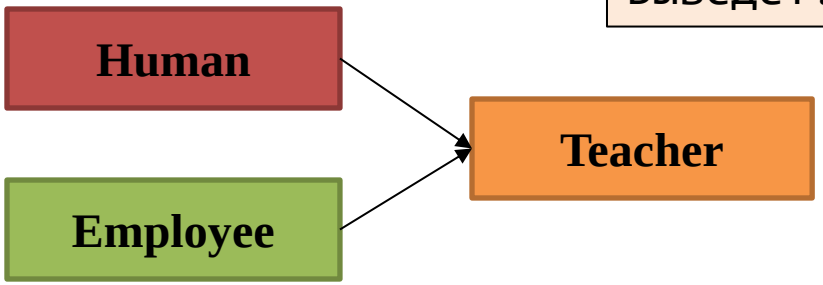
```
# Первый базовый класс, моделирующий человека
class Human:
    def __init__(self, name, age):
        self._name = name  # У него есть имя
        self._age = age    # и возраст

    def info(self):  # Выводим их на экран
        print(self._name, self._age, end = ' ')

# Второй базовый класс, моделирующий работника
class Employee:
    def __init__(self, post, wage):
        self._post = post  # У него есть должность
        self._wage = wage  # и зарплата

    def info(self):  # Выводим их на экран
        print(self._post, self._wage, end = ' ')
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример множественного наследования (продолжение)

```
# Класс Teacher наследует классы Human и Employee
class Teacher(Human, Employee):
    def __init__(self, name, age, post, wage, grade):
        # Явно вызываем конструкторы каждого базового класса
        # Обратите внимание на передачу параметра self
        Human.__init__(self, name, age)
        Employee.__init__(self, post, wage)
        self._grade = grade

    def info(self):
        Human.info(self) # при совпадении имён нужно явно
        Employee.info(self) # указать базовый класс
        print(self._grade)

if __name__ == '__main__':
    t = Teacher('Петров П.П.', 55, 'Доцент ИУС', 40000, 4)
    t.info()
```

```
Выведет: Петров П.П. 55 Доцент ИУС 40000 4
```



© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева